Brightloop Software: Subscription & Recurring Billing ERP

Contract-First Implementation Plan, Unified Schemas, Role Specifications, and Judge-Breaker Edge Cases

1. Executive Overview & Domain Mental Model

Brightloop Software sells B2B software subscriptions billed on recurring schedules (daily, weekly, monthly, or yearly), optional add-on modules that customers toggle on or off, and product variants that alter pricing. Today, Brightloop tracks subscriptions in spreadsheets, types invoices by hand, matches bank payments manually, and misses renewals. Our objective is to build the **Subscription & Recurring Billing Management ERP** that automates the entire contract and financial lifecycle.

> **Crucial Scope Clarification (What We Are & Are Not Building):** We are **NOT** building the actual software tools that Brightloop sells (such as a CRM or Inventory tool). In our system, a "Product", "Variant", or "Add-on" is strictly a catalog record in our database with pricing, cost, tax, and billing rules. Our platform manages the commercial engine: **Catalog Setup → Quotation Templates → Subscription Lifecycle → Automated Billing Day Invoicing → Payment Ledger → Sub-2-Second Finance Analytics.**

Concrete End-to-End Business Example

Concept	Brightloop Real-World Scenario	How Our System Processes It
Base Product	Brightloop Cloud ERP (Base Sales Price: ₹2,000, Cost Price: ₹500, Type: Service)	Stored in `products` table. Admin configures base sales price, cost price, product type, and notes.
Product Variant	Customer chooses the **Brand: Odoo** variant, which adds **+560** to the unit price.	Stored in `product_variants` (`price_extra = 560.00`). Effective unit price automatically becomes ₹2,560.00.
Add-on Product	WhatsApp Automated Alerts (₹300/month, Type: Add-on).	Customer toggles it on/off in their portal, adding/removing a `subscription_lines` row for future billing cycles.
Billing Plan	Annual Enterprise Plan (Billed Yearly, Min Qty: 5 seats, Pausable: Yes, Early Close: No).	Stored in `plans` . Enforces minimum quantity of 5 and controls which lifecycle buttons are enabled.

Concept	Brightloop Real-World Scenario	How Our System Processes It
Discount & Tax	10% Off for ≥10 seats (Max 50 uses, valid until Dec 31) + 18% GST Tax Rule.	Pricing engine validates expiry date, min seats, and remaining uses before deducting 10% and adding 18% GST.

2. Recommended Technology Stack & Rationale

Layer	Selected Stack	Why This Wins the Odoo Hackathon
Backend API	Python 3.11 + FastAPI + Pydantic v2	Odoo is written in Python. FastAPI provides automatic Swagger UI (`/docs`) for live judge inspection and native Pydantic schema validation.
Database & ORM	PostgreSQL + SQLAlchemy 2.0	Enterprise relational integrity, composite B-Tree indexes for <2s dashboard speed on 10,000+ rows, transaction savepoints, and `UNIQUE` constraints to prevent double-billing.
Financial Math	Python `decimal.Decimal` + `python-dateutil`	Eliminates IEEE-754 floating-point penny drift and handles exact calendar month/year increments (`Jan 31 + 1 month = Feb 28`).
Frontend UI	React (Vite) + Tailwind CSS + shadcn/ui + Recharts	Rapidly builds Odoo-style status pipeline ribbons, quotation line tables, role-based portals, printable PDF invoices, and revenue charts.

3. Zero-Conflict 4-Person Team Architecture

To respect our team division—**Member 1 (Backend Core & PostgreSQL DB)**, **Member 2 (Features & Business Logic Engine)**, **Member 3 (Frontend UI)**, and **Member 4 (Integration, Wiring & Scale Testing)**—without Git merge conflicts or idle waiting, we enforce strict folder isolation and a **Contract-First Workflow**. Each member works strictly inside their assigned module boundaries and communicates through the shared field contracts defined in Sections 4 and 5.

Owner & Role	Workspace Folder	Key Files Owned	Handoff Interface
Member 1 Backend Core & DB	`/backend/core/`	`database.py` , `models.py` , `schemas.py` , `auth.py` , `catalog_routes.py`	Provides PostgreSQL tables, JWT role guards, and Catalog CRUD to Member 4.
Member 2 Features Engine	`/backend/engine/`	`pricing.py` , `state_machine.py` , `billing_job.py`	Provides pure calculation & rule validation functions to Member 4.

Owner & Role	Workspace Folder	Key Files Owned	Handoff Interface
Member 3 Frontend UI	`/frontend/src/`	`pages/*`, `components/*`, `mocks/*`	Builds screens against shared JSON contracts; exposes `services/api.js` for Member 4.
Member 4 Integration & Scale	`/backend/api/` & `/frontend/src/services/`	`main.py`, `sub_routes.py`, `invoice_routes.py`, `analytics.py`, `seed_benchmark.py`	Connects Member 1 DB + Member 2 Engine + Member 3 UI; verifies <2s speed.

4. Unified PostgreSQL Database Schema Specification

All 12 PostgreSQL tables must be initialized in Hour 1 using these exact table names, field names, data types, and indexing rules. Currency values use `NUMERIC(12, 2)` to prevent floating-point rounding errors, and all dates use standard `DATE` (`YYYY-MM-DD`).

4.1 Identity, Catalog, Taxes & Discounts Tables

Table Name	Required Columns & Types	Constraints, Indexes & Business Rules
`users`	`id` (PK), `email` (VARCHAR, Unique), `password_hash` (VARCHAR), `full_name` (VARCHAR), `role` (ENUM: `ADMIN`, `STAFF`, `CUSTOMER`), `reset_token` (VARCHAR, Nullable), `created_at` (TIMESTAMP)	Unique index on `email`. Only `ADMIN` can create `STAFF` accounts. Password must be >8 chars with upper, lower, and special char.
`products`	`id` (PK), `name` (VARCHAR), `product_type` (ENUM: `SERVICE`, `ADDON`), `sales_price` (NUMERIC 12,2), `cost_price` (NUMERIC 12,2), `notes` (TEXT), `is_active` (BOOLEAN)	Base commercial catalog item. Managed exclusively by `ADMIN`.
`product_variants`	`id` (PK), `product_id` (FK → `products.id`), `attribute_name` (VARCHAR), `attribute_value` (VARCHAR), `price_extra` (NUMERIC 12,2)	Example: `attribute_name = "Brand"`, `attribute_value = "Odoo"`, `price_extra = 560.00`. Cascades on product delete.
`tax_rules`	`id` (PK), `name` (VARCHAR), `rate_percent` (NUMERIC 5,2), `is_active` (BOOLEAN)	Example: `GST 18%` (`18.00`). Calculated automatically on taxable line amounts.

Table Name	Required Columns & Types	Constraints, Indexes & Business Rules
`discount_rules`	`id` (PK), `name` (VARCHAR), `discount_type` (ENUM: `PERCENT`, `FIXED`), `value` (NUMERIC 12,2), `scope` (ENUM: `PRODUCT`, `SUBSCRIPTION`), `min_qty` (INT), `min_purchase` (NUMERIC 12,2), `valid_from` (DATE), `valid_until` (DATE), `max_uses` (INT, Nullable), `current_uses` (INT)	Admin-only creation. Automatically stops applying on invoices once `today > valid_until` or `current_uses >= max_uses`.

4.2 Plans, Templates & Live Subscription Tables

Table Name	Required Columns & Types	Constraints, Indexes & Business Rules
`plans`	`id` (PK), `name` (VARCHAR), `billing_frequency` (ENUM: `DAILY`, `WEEKLY`, `MONTHLY`, `YEARLY`), `plan_price` (NUMERIC 12,2), `min_qty` (INT), `available_from` (DATE), `available_until` (DATE), `auto_close` (BOOL), `can_close_early` (BOOL), `can_pause` (BOOL), `can_renew` (BOOL)	Defines recurring schedule and the four boolean flags governing allowed subscription lifecycle actions.
`quotation_templates`	`id` (PK), `name` (VARCHAR), `plan_id` (FK → `plans.id`), `validity_days` (INT)	Allows sales staff to pre-fill a quotation in 1 click with preset plan and validity window.
`quotation_template_lines`	`id` (PK), `template_id` (FK → `quotation_templates.id`), `product_id` (FK → `products.id`), `variant_id` (FK → `product_variants.id`, Nullable), `quantity` (INT)	Default product and variant lines copied into a new subscription when a template is selected.

Table Name	Required Columns & Types	Constraints, Indexes & Business Rules
`subscriptions`	`id` (PK), `sub_number` (VARCHAR, Unique), `customer_id` (FK → `users.id`), `plan_id` (FK → `plans.id`), `discount_id` (FK → `discount_rules.id`, Nullable), `status` (ENUM: `DRAFT`, `QUOTATION`, `CONFIRMED`, `ACTIVE`, `PAUSED`, `CLOSED`), `start_date` (DATE), `next_billing_date` (DATE), `expiry_date` (DATE, Nullable), `paused_at` (DATE, Nullable), `payment_terms_days` (INT)	**Composite B-Tree Index:** `(status, next_billing_date)` for instant Billing Day lookup and active count queries.
`subscription_lines`	`id` (PK), `subscription_id` (FK → `subscriptions.id`), `product_id` (FK → `products.id`), `variant_id` (FK, Nullable), `tax_id` (FK, Nullable), `discount_id` (FK, Nullable), `quantity` (INT), `unit_price` (NUMERIC 12,2)	Stores active contracted items (including customer-toggled add-ons) and effective unit price (`sales_price + price_extra`).

4.3 Immutable Invoicing & Payment Ledger Tables

Table Name	Required Columns & Types	Constraints, Indexes & Business Rules
`invoices`	`id` (PK), `invoice_number` (VARCHAR, Unique), `subscription_id` (FK → `subscriptions.id`), `customer_id` (FK → `users.id`), `billing_period_key` (VARCHAR), `status` (ENUM: `DRAFT`, `CONFIRMED`, `PARTIALLY_PAID`, `PAID`, `CANCELLED`), `issue_date` (DATE), `due_date` (DATE), `subtotal` (NUMERIC 12,2), `total_discount` (NUMERIC 12,2), `total_tax` (NUMERIC 12,2), `grand_total` (NUMERIC 12,2), `amount_paid` (NUMERIC 12,2)	**Unique Constraint:** `(subscription_id, billing_period_key)` prevents double-billing. **Composite Index:** `(status, due_date)` for <2s overdue invoice filtering.

Table Name	Required Columns & Types	Constraints, Indexes & Business Rules
`invoice_lines`	`id` (PK), `invoice_id` (FK → `invoices.id`), `product_name_snapshot` (VARCHAR), `variant_name_snapshot` (VARCHAR, Nullable), `quantity` (INT), `unit_price_snapshot` (NUMERIC 12,2), `discount_amount` (NUMERIC 12,2), `tax_percent_snapshot` (NUMERIC 5,2), `tax_amount` (NUMERIC 12,2), `line_total` (NUMERIC 12,2)	**Snapshot Pattern:** Copies text names, prices, discounts, and tax rates at billing time so future catalog edits never alter past invoices.
`payments`	`id` (PK), `invoice_id` (FK → `invoices.id`), `payment_method` (ENUM: `UPI`, `BANK_TRANSFER`, `CARD`, `CASH`), `amount` (NUMERIC 12,2), `payment_date` (DATE), `note` (TEXT, Nullable)	Indexed on `invoice_id`. Multiple partial payments sum into `invoices.amount_paid` until `grand_total` is reached.

5. Master Shared Data Contracts (Frontend ↔ API ↔ Engine)

All four teammates must use these exact field names across API payloads, Engine dictionaries, and React state. Dates are always formatted as `"YYYY-MM-DD"` strings and currency figures as two-decimal numbers.

Contract Name & Endpoint	Top-Level Fields	Nested Arrays / Objects
Contract A: Subscription Object `GET /api/subscriptions/{id}`	`id`, `sub_number`, `customer_id`, `customer_name`, `plan_id`, `plan_name`, `billing_frequency`, `status`, `start_date`, `next_billing_date`, `expiry_date`, `paused_at`	**`plan_permissions`**: `auto_close`, `can_close_early`, `can_pause`, `can_renew` **`lines[]`**: `product_id`, `product_name`, `variant_name`, `quantity`, `unit_price`, `discount_amount`, `discount_applied`, `tax_percent`, `tax_amount`, `line_total` **`summary`**: `subtotal`, `line_discount_total`, `subscription_discount`, `total_discount`, `total_tax`, `grand_total`

Contract Name & Endpoint	Top-Level Fields	Nested Arrays / Objects
Contract B: Invoice & Payments Object `GET /api/invoices/{id}`	`id`, `invoice_number`, `subscription_id`, `sub_number`, `customer_name`, `billing_period_key`, `status`, `issue_date`, `due_date`, `is_overdue`, `subtotal`, `total_discount`, `total_tax`, `grand_total`, `amount_paid`, `amount_due`	**`lines[]`**: `product_name_snapshot`, `variant_name_snapshot`, `quantity`, `unit_price_snapshot`, `discount_amount`, `tax_percent_snapshot`, `tax_amount`, `line_total` **`payments[]`**: `id`, `payment_method`, `amount`, `payment_date`, `note`
Contract C: Finance Lead Summary `GET /api/analytics/summary`	`query_time_ms`, `active_subscriptions_count`, `mrr_collected`, `total_outstanding_owed`, `overdue_invoices_count`	**`overdue_invoices[]`**: `invoice_id`, `invoice_number`, `customer_name`, `amount_due`, `due_date`, `days_overdue`

6. Detailed Role-by-Role Implementation Manual

Member 1: Backend Core, Security & Catalog CRUD (`/backend/core/`)

Mission: Initialize the PostgreSQL database, enforce authentication and password security rules, enforce Role-Based Access Control (`ADMIN`, `STAFF`, `CUSTOMER`), and provide CRUD endpoints for all master catalog entities.

- **Strict Password Validation:** Must enforce more than 8 characters (9+ characters), at least 1 uppercase letter, 1 lowercase letter, and 1 special character. Reject duplicate emails immediately.
- **Role-Based Access Control (RBAC):** Public signup creates `CUSTOMER` accounts. Only an authenticated `ADMIN` can create `STAFF` users or configure Products, Variants, Plans, Taxes, and Discount Rules. `STAFF` can manage quotations, subscriptions, invoices, and payments. `CUSTOMER` can only access their own records.
- **Password Reset Flow:** Generate a reset token stored on `users.reset_token` and send/log the reset URL so forgotten passwords can be recovered cleanly.
- **Catalog Endpoints to Deliver:** Full CRUD for Products & Variants (supporting the `+560` Odoo brand delta), Plans, Tax Rules, Discount Rules, and Quotation Templates.

Member 2: Features & Business Logic Engine (`/backend/engine/`)

Mission: Build three pure business-logic modules that accept flat parameters and output dictionaries matching Section 5. Zero database or UI dependencies so Member 4 can plug them directly into API endpoints.

Engine Module & Function	Required Inputs	Exact Business Rules & Math	Returned Output Dictionary
1. Pricing Engine `pricing.py` `calculate_line_item()`	`base_price`, `variant_extra`, `quantity`, `tax_percent`, discount fields (`type`, `value`, `min_qty`, `min_purchase`, `uses_left`, `valid_from`, `valid_until`), `today_str`	1. *Unit = Base + VariantExtra* 2. *Gross = Unit × Qty* 3. Verify discount date window, `Qty ≥ min_qty`, `Gross ≥ min_purchase`, and `uses_left > 0`. 4. *Taxable = Gross − Discount* 5. *Tax = Taxable × (TaxRate / 100)*	`unit_price`, `gross_amount`, `discount_amount`, `discount_applied` (bool), `taxable_amount`, `tax_amount`, `line_total`
2. State Machine `state_machine.py` `check_subscription_transition()`	`current_status`, `target_status`, plan flags (`can_pause`, `can_close_early`, `can_renew`), `next_billing_date`, `expiry_date`, `paused_at`, `today_str`	1. Enforce pipeline: `DRAFT → QUOTATION → CONFIRMED → ACTIVE → PAUSED / CLOSED`. 2. Block pause, early close, or renewal if plan boolean flag is `false`. 3. On `PAUSED → ACTIVE`, shift `next_billing_date` and `expiry_date` forward by paused days.	`allowed` (bool), `next_status`, `next_billing_date`, `expiry_date`, `paused_at`, `error` (string or null)
3. Billing Evaluator `billing_job.py` `evaluate_subscription_for_billing()`	`sub_number`, `status`, `billing_frequency`, `next_billing_date`, `expiry_date`, `auto_close`, `today_str`, `existing_period_keys`	1. If `today_str ≥ expiry_date` and `auto_close == true`, return `AUTO_CLOSE`. 2. Check `status == ACTIVE` and `next_billing_date ≤ today_str`. 3. Build key `"{sub_number}:{next_billing_date}"`; skip if already in `existing_period_keys`. 4. Advance next cycle by exact calendar delta (Day/Week/Month/Year).	`action` (`BILL`, `AUTO_CLOSE`, or `SKIP`), `billing_period_key`, `next_billing_date`, `new_status`, `reason`

Member 3: Frontend ERP Workspace, Portal & PDF UI (`/frontend/`)

Mission: Build a responsive, Odoo-inspired interface using the shared contracts from Section 5 so UI development proceeds in parallel without waiting on backend routes.

- **View 1 — Auth & Dynamic Role Shell:** Signup form with real-time password requirement indicators (>8 chars, uppercase, lowercase, special character) + Forgot Password modal. Role-aware sidebar showing Admin Config + Staff Operations + Finance Analytics to `ADMIN`, Day-to-Day Operations to `STAFF`, and Self-Service Portal to `CUSTOMER`.
- **View 2 — Admin Catalog & Rules Studio:** Clean forms and tables for Products & Variants (showing `+560` extra price badges), Plans (with toggles for Auto-Close, Early Close, Pause, Renew), Tax Rules, Discount Rules, and Quotation Templates.
- **View 3 — Staff Subscription & Quotation ERP:** "Create from Template" selector that auto-populates plan and product lines. Interactive status chevron bar (`Draft → Quotation → Confirmed → Active → Paused → Closed`) where action buttons dynamically disable if `plan_permissions` disallow them.
- **View 4 — Billing Simulator Bar, Invoice View & Payment Modal:** Header bar with a date picker (`Simulate Billing Date`) and a "**Run Billing Day Batch**" button for live demoing. Invoice detail screen with a 1-click **Print / Download PDF** stylesheet and a "**Record Payment**" modal (Method, Amount, Date, Note).
- **View 5 — Customer Self-Service Portal & Finance Lead Dashboard:** Customers view their own subscriptions, toggle add-ons, pause/renew (if allowed by plan), and print invoices. Finance Lead sees 4 KPI cards, overdue chase table with date/status filters, and a live `⚡ Query Time: XX ms` speed badge.

Member 4: Integration, Wiring, Analytics SQL & Scale Testing (`/backend/api/` & `/frontend/src/services/`)

Mission: Own the bridge between PostgreSQL, Engine, and Frontend continuously from Hour 1—ensuring zero last-minute integration panic and proving the <2-second performance requirement on thousands of rows.

- **Continuous Integration Schedule:** In Hours 1–3, lock `models.py` and `schemas.py` with Member 1 and generate `mocks.json` for Member 3. In Hours 4–12, wire Subscription and Invoice endpoints by reading Member 1's PostgreSQL models and passing fields into Member 2's `engine/` functions. In Hours 13–20, swap Member 3's mock calls in `services/api.js` with live API requests.
- **Atomic Batch Billing Execution:** Process each due subscription inside a PostgreSQL savepoint and increment `discount_rules.current_uses` atomically so a single invalid record never blocks a batch of 500 due invoices.
- **Sub-50ms PostgreSQL Finance Dashboard:** Execute a single indexed SQL aggregation query (`COUNT` and `SUM` with `CASE WHEN` filters on `invoices` and `subscriptions`) and attach an `X-Process-Time` response header to prove sub-2-second performance live.

7. Judge-Breaker Edge Cases & Reliability Checklist

Edge Case	What Breaks in Naive Hackathon Projects	How Our Architecture Solves It
Double-Billing on Billing Day	Clicking "Generate Invoices" twice bills every customer twice for the same month.	`UniqueConstraint(subscription_id, billing_period_key)` + engine period-key check guarantees 100% idempotency.
Retroactive Price / Tax Mutation	Admin updates product price from ₹2,000 to ₹2,500; old paid invoices change their totals.	`InvoiceLine` snapshots product name, variant, unit price, discount, and tax rate at the moment of billing.
Discount Expiry & Usage Exhaustion	Discounts keep applying forever or exceed `max_uses` during batch billing.	`calculate_line_item()` checks `today_str <= valid_until` and `uses_left > 0` on every cycle; drops expired discounts automatically.
Unfair Billing During Pause	Customer pauses for 12 days, resumes, and gets billed on the old date anyway.	`check_subscription_transition()` shifts `next_billing_date` and `expiry_date` forward by exact paused days.
Partial & Split Payments	System marks an invoice `PAID` when only ₹5,000 of ₹20,000 is received.	Tracks `amount_paid` vs `grand_total`; transitions through `PARTIALLY_PAID` until `amount_due == 0`.
<2s Dashboard with 5,000+ Subscriptions	Fetching 15,000 ORM objects into Python memory takes 5+ seconds and times out.	Composite B-Tree indexes + server-side SQL `SUM/COUNT` aggregation responds in <45ms.

8. Hour-by-Hour Parallel Execution & Git Sync Plan

Phase	Member 1 (Core/DB)	Member 2 (Engine)	Member 3 (Frontend)	Member 4 (Integration)
Hours 1–3: Foundation	Commit `models.py` & `database.py`; build JWT Auth & password regex.	Build & unit-test `pricing.py` (+560 variant, discounts, taxes).	Setup React + Tailwind + Role Sidebar + Auth UI with live password checklist.	Commit `schemas.py` & frontend `mocks.json`; setup FastAPI `main.py`.
Hours 4–9: Core Modules	Build Catalog CRUD APIs (Products, Variants, Plans, Taxes, Discounts, Templates).	Build & test `state_machine.py` and `billing_job.py` with edge cases.	Build Admin Catalog screens & Staff Subscription form with status ribbon.	Wire Subscription & Template APIs calling Member 2's `pricing` & `state_machine`.

Phase	Member 1 (Core/DB)	Member 2 (Engine)	Member 3 (Frontend)	Member 4 (Integration)
Hours 10–16: Billing & Portal	Add role guards & password reset token flow; assist Member 4 on queries.	Write edge-case verification suite (pause shift, discount cap, idempotency).	Build Invoice view, Printable PDF layout, Payment modal & Customer Portal.	Wire Batch Billing Day endpoint, Payment ledger & connect React Catalog/Sub APIs.
Hours 17–24: Scale & Polish	Freeze PostgreSQL indexes; verify connection pooling and role security rules.	Help run `seed_benchmark.py` (5,000 subs) & verify all accounting math.	Build Finance Lead Dashboard + Time-Travel Simulator bar; UI polish.	Wire Finance SQL endpoint (<50ms), connect remaining UI screens & rehearse demo.